

Practical Task 1 — Interactive Web Application

Objective

Build a responsive web application using:

- Semantic HTML5
- CSS3
- Flexbox or CSS Grid
- JavaScript
- DOM manipulation
- Events
- Form validation
- Filtering and sorting
- localStorage
- REST API
- fetch()
- JSON
- async/await
- Loading and error states

Project: GitHub Repository Explorer

Create a web application that allows users to search for a GitHub user and display their repositories.

The application should have a clean, responsive interface and dynamically load data from the GitHub REST API.

Part 1 — Project Structure

Create the following project structure:

```
github-explorer/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── script.js  
└── assets/
```

Do not put all HTML, CSS, and JavaScript in one file.

Part 2 — Semantic HTML

Use semantic HTML5 elements.

Your page should contain:

- `<header>`
- `<nav>`
- `<main>`
- `<section>`
- `<footer>`

Create a navigation menu containing:

- Home
- Search
- About

The navigation should work correctly.

Part 3 — Search Form

Create a search form containing:

- GitHub username input
- Search button

Example:

GitHub username: [] [Search]

Add HTML and JavaScript validation.

The application should not send an API request if the input is empty.

Display an appropriate validation message.

Part 4 — REST API

Use the GitHub REST API to retrieve repository information.

Use JavaScript `fetch()` to send a request.

The application should retrieve information such as:

- Repository name
- Description
- Programming language
- Number of stars
- Repository URL

Use `async/await` to handle the asynchronous request.

Part 5 — JSON and Dynamic Rendering

Process the JSON response from the API.

Do not write repository cards manually in HTML.

Create them dynamically using JavaScript.

The number of repository cards should depend on the API response.

Part 6 — Loading State

While the application is waiting for the API response, display a loading message.

Example:

```
Loading repositories...
```

The loading message should disappear after the request is completed.

Part 7 — Error Handling

Handle possible errors.

For example:

- Empty username
- User does not exist
- API request fails
- No repositories found

Display a meaningful message to the user.

Example:

User not found.

Do not leave the page blank when an error occurs.

Part 8 — Filtering

Add a filter for programming languages.

For example:

All | JavaScript | Python | Java | Other

When the user selects a language, display only repositories that match that language.

The filtering must be performed using JavaScript.

Part 9 — Sorting

Add at least two sorting options.

For example:

Sort by:
- Repository name
- Stars

The user should be able to change the sorting order dynamically.

Part 10 — localStorage

Use `localStorage` to remember the last searched GitHub username.

When the user opens the application again, display the last searched username in the search field.

Optional:

Automatically perform the search for the saved username.

Part 11 — Responsive Design

The application must work correctly on:

- Desktop
- Tablet
- Mobile

Use CSS Flexbox or CSS Grid.

Add at least one media query.

Example:

```
@media (max-width: 600px) {  
  .repository-grid {  
    grid-template-columns: 1fr;  
  }  
}
```

Part 12 — UI Requirements

The application should have:

- Clear typography
- Consistent spacing
- Buttons with hover effects
- Repository cards
- Responsive layout
- Clear loading message
- Clear error message
- User-friendly form